

Kubernetes

Interview Bible — From Fundamentals to Senior-Level Mastery

DevOps • SRE • Cloud Operations

A complete reference — architecture, core objects, networking, storage, security, GitOps, troubleshooting, common errors, tricky questions & expected Q&A for candidates with 5+ years of experience

Table of Contents

1. Kubernetes Architecture & Fundamentals
2. Core Workload Objects
3. Networking (Services, Ingress, DNS, Network Policy)
4. Storage (Volumes, PV/PVC, StorageClass, CSI)
5. Configuration & Secrets
6. Scheduling, Resource Management & Autoscaling
7. Security & RBAC
8. Helm & Kustomize
9. GitOps Integration (ArgoCD/Flux) & CI/CD
10. Observability & Health Checks
11. Common Errors & Troubleshooting Playbook
12. Tricky Questions & Traps
13. Expected Interview Q&A (Basic → Advanced)
14. System Design Style Questions
15. Behavioral / Experience-Based Questions
16. Quick Reference Cheat Sheet (kubect!)

1. Kubernetes Architecture & Fundamentals

1.1 What Is Kubernetes

Kubernetes (K8s) is a container orchestration platform that automates deployment, scaling, networking, and lifecycle management of containerized applications, presenting a declarative API: you describe the **desired state** and controllers continuously reconcile actual state toward it.

1.2 Control Plane Components

Component	Role
kube-apiserver	Front door to the cluster; validates and processes all REST API requests; only component that talks directly to etcd
etcd	Distributed key-value store holding all cluster state (the "source of truth")
kube-scheduler	Assigns newly created Pods to Nodes based on resource requirements, constraints, affinity/anti-affinity
kube-controller-manager	Runs core controllers (Node, ReplicaSet, Endpoint, Job, etc.) that reconcile desired vs actual state
cloud-controller-manager	Integrates with the underlying cloud provider (load balancers, node lifecycle, routes)

1.3 Node Components

Component	Role
kubelet	Agent on every node; ensures containers described in PodSpecs are running and healthy
kube-proxy	Maintains network rules on nodes to implement the Service abstraction (iptables/IPVS)
Container runtime	containerd or CRI-O (Docker Engine itself is no longer used directly since dockershim removal) — runs the actual containers via the Container Runtime Interface (CRI)

1.4 The Reconciliation Loop Senior favorite

Every controller follows the same pattern: **watch** the API server for the desired state, **compare** to observed actual state, and **act** to close the gap — repeatedly, forever. This is why Kubernetes is "self-healing": if a Pod dies, the ReplicaSet controller notices the actual replica count is below desired and creates a replacement, with no human or pipeline intervention needed.

1.5 API Objects & the Declarative Model

Everything in Kubernetes is a resource with a spec (desired state) and a status (observed state), expressed as YAML/JSON manifests applied via `kubectl apply` or a controller (Helm, ArgoCD). Custom Resource Definitions (CRDs) let you extend the API with your own object types, reconciled by custom controllers/operators — the foundation of the operator pattern (e.g., Prometheus Operator, cert-manager, ArgoCD's own Application CRD).

1.6 Namespaces

Namespaces provide logical isolation (naming scope, RBAC boundary, resource quota boundary) within one cluster — not network isolation by default (that requires NetworkPolicy) and not a security boundary as strong as separate clusters (some resources, like Nodes and PersistentVolumes, are cluster-scoped and not namespaced at all).

2. Core Workload Objects

2.1 Pod

The smallest deployable unit — one or more containers that share network namespace (same IP/port space) and can share storage volumes. Pods are meant to be **ephemeral**: you almost never create a bare Pod directly in production; a higher-level controller manages Pod lifecycle instead.

2.2 ReplicaSet & Deployment

A **ReplicaSet** ensures a specified number of identical Pod replicas are running. A **Deployment** manages ReplicaSets and adds rolling update/rollback capability — you almost always use a Deployment, not a bare ReplicaSet, since it's what gives you `kubectl rollout undo` and controlled rollout strategy.

```
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1           # extra pods allowed above desired count during rollout
      maxUnavailable: 0    # pods allowed to be unavailable during rollout
  template:
    spec:
      containers:
        - name: app
          image: myapp:1.4.2
          readinessProbe: { httpGet: { path: /healthz, port: 8080 }, initialDelaySeconds: 5 }
          resources:
            requests: { cpu: 200m, memory: 256Mi }
            limits: { cpu: 500m, memory: 512Mi }
```

2.3 StatefulSet

For workloads needing stable, unique network identities and stable storage across restarts (databases, Kafka, Elasticsearch): Pods get predictable ordinal names (`app-0` , `app-1` ...), stable DNS via a headless Service, and each Pod's PVC persists across rescheduling and is not shared with other replicas. Scaling up/down and rolling updates happen in strict ordinal order by default.

2.4 DaemonSet

Ensures exactly one Pod copy runs on every (or a selected subset of) node — used for node-level agents: log collectors (Fluentd/Fluent Bit), monitoring agents (node-exporter), CNI plugins, and security/scanning agents.

2.5 Job & CronJob

Job runs a Pod to completion (not restarted after success), useful for batch/one-off tasks (migrations, report generation); supports parallelism and retry backoff. **CronJob** schedules Jobs on a cron schedule — commonly used for scheduled maintenance tasks, backups, and cleanup.

2.6 Deployment vs StatefulSet vs DaemonSet — Choosing Senior favorite

Need	Object
Stateless, interchangeable replicas (web/API servers)	Deployment
Stable identity/storage per instance (databases, message brokers)	StatefulSet
Exactly one Pod per node (agents, log shippers)	DaemonSet
Run-to-completion batch task	Job / CronJob

3. Networking

3.1 Service Types

Type	Description
ClusterIP (default)	Internal-only virtual IP, reachable only within the cluster
NodePort	Exposes the service on a static port on every node's IP; mainly for dev/testing or on-prem without a cloud LB
LoadBalancer	Provisions an external cloud load balancer (via cloud-controller-manager) pointing at the service
ExternalName	Maps a service name to an external DNS name (CNAME-like), no proxying involved
Headless (<code>clusterIP: None</code>)	No virtual IP; DNS returns individual Pod IPs directly — used by StatefulSets for stable per-Pod addressing

3.2 kube-proxy & Service Discovery

Services get a stable virtual IP; kube-proxy programs iptables/IPVS rules on every node to load-balance traffic to that IP across the matching Pod endpoints, which are tracked automatically via label selectors and updated as Pods come and go. CoreDNS provides in-cluster DNS so services are reachable by name (`service-name.namespace.svc.cluster.local`).

3.3 Ingress & Ingress Controllers

A **Service** load-balances at L4; an **Ingress** resource defines L7 HTTP(S) routing rules (host/path-based routing, TLS termination) that require a separately-installed **Ingress Controller** (NGINX Ingress, Traefik, HAProxy, or cloud-native ALB/GKE ingress controllers) to actually implement the routing — the Ingress object alone does nothing without a controller watching it.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: app-svc, port: { number: 80 } }
  tls:
    - hosts: [app.example.com]
      secretName: app-tls
```

3.4 Gateway API Senior favorite

The newer, more expressive successor to Ingress (role-oriented: infra teams manage Gateways, app teams manage Routes; supports advanced traffic splitting/header-based routing natively) — increasingly used for progressive delivery (canary traffic splits) without needing a service mesh.

3.5 Network Policies

By default, all Pods can reach all other Pods (flat network) — a NetworkPolicy is required to restrict traffic (default-deny + explicit allow rules), and it only takes effect if the cluster's CNI plugin supports NetworkPolicy enforcement (e.g., Calico, Cilium; some CNIs like basic Flannel don't enforce policies at all).

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector: { matchLabels: { app: frontend } }
      ports: [{ port: 8080 }]
```

3.6 Service Mesh (Istio/Linkerd)

Adds a sidecar proxy to every Pod for mTLS between services, fine-grained traffic management (canary/mirroring), retries/circuit-breaking,

and rich telemetry — independent of the application code. Trade-off: added latency/resource overhead and operational complexity, so it's usually adopted once traffic-management/security needs outgrow what plain Ingress + NetworkPolicy can provide.

4. Storage

4.1 Volumes vs PersistentVolumes

A basic `volume` is tied to the Pod's lifecycle (deleted when the Pod is deleted) — fine for scratch space or sharing files between containers in the same Pod. A **PersistentVolume (PV)** is a cluster-level storage resource with a lifecycle independent of any Pod; a **PersistentVolumeClaim (PVC)** is a namespaced request for storage that binds to a matching PV.

4.2 StorageClass & Dynamic Provisioning

A **StorageClass** defines a "class" of storage (e.g., SSD vs HDD, replication factor) and a provisioner; when a PVC references a StorageClass, the provisioner automatically creates a matching PV on demand (dynamic provisioning) — the modern default, replacing manually pre-creating PVs.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data-pvc }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: fast-ssd
  resources: { requests: { storage: 10Gi } }
```

4.3 Access Modes

Mode	Meaning
ReadWriteOnce (RWO)	Mounted read-write by a single node (most block storage: EBS, GCE PD, Azure Disk)
ReadOnlyMany (ROX)	Mounted read-only by many nodes
ReadWriteMany (RWX)	Mounted read-write by many nodes simultaneously (needs a shared filesystem: NFS, EFS, Azure Files, CephFS)
ReadWriteOncePod (RWOP)	Restricts mount to a single Pod (not just a single node) — stronger guarantee for exclusive-write workloads

Trap: RWO is per-*node*, not per-Pod — multiple Pods on the same node can still mount an RWO volume simultaneously; use RWOP if you truly need single-Pod exclusivity.

4.4 CSI (Container Storage Interface)

The standard plugin interface that lets any storage vendor (cloud block/file storage, Ceph, Portworx, etc.) integrate with Kubernetes without in-tree code changes — virtually all storage integrations today are CSI drivers, since the older in-tree volume plugins are deprecated/removed.

4.5 Reclaim Policies

`Retain` keeps the underlying storage after the PVC is deleted (manual cleanup required — safest for critical data); `Delete` automatically deletes the underlying storage resource when the PVC is deleted (default for most dynamically provisioned classes — convenient but risks accidental data loss if misconfigured for production databases).

4.6 StatefulSets and Storage

StatefulSets use `volumeClaimTemplates` to give each replica its own dedicated PVC that persists across Pod rescheduling — Pod `app-0` always reattaches to its own PVC even if rescheduled to a different node, which is essential for stateful workloads like databases.

5. Configuration & Secrets

5.1 ConfigMap

Stores non-sensitive configuration as key-value pairs, injectable as environment variables or mounted files. Mounted-file ConfigMaps update automatically in the Pod's filesystem when changed (with propagation delay), but env-var injections do NOT update without a Pod restart — a very common interview trap and real-world gotcha.

5.2 Secret

Same shape as a ConfigMap but intended for sensitive data. **Critical trap:** Secrets are only *base64-encoded*, not encrypted, by default — anyone with API/etcd access can trivially decode them. Real encryption at rest requires enabling **encryption at rest** (EncryptionConfiguration with a KMS provider) at the API server level; without it, "Secret" is really just "obfuscated ConfigMap".

5.3 Managing Secrets Safely in GitOps

Since Secrets shouldn't be committed to Git in plaintext (even base64 isn't safe), two common patterns: **Sealed Secrets** (Bitnami) encrypts a Secret client-side into a SealedSecret CRD that only the in-cluster controller can decrypt, safe to commit to Git; **External Secrets Operator** syncs real secrets from an external manager (Vault, AWS Secrets Manager, Azure Key Vault) into native K8s Secrets at runtime, so Git never stores secret material at all — only a reference to where it lives externally.

5.4 Injecting Config into Pods

```
env:
- name: LOG_LEVEL
  valueFrom:
    configMapKeyRef: { name: app-config, key: log-level }
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef: { name: db-secret, key: password }
volumeMounts:
- name: config-volume
  mountPath: /etc/config
volumes:
- name: config-volume
  configMap: { name: app-config }
```

5.5 Immutable ConfigMaps/Secrets

Setting `immutable: true` prevents accidental in-place edits (which can cause confusing partial-rollout behavior since different Pods may read the change at different times); instead, changes require creating a new ConfigMap/Secret with a new name and updating the Deployment reference — which also naturally triggers a proper rolling restart.

6. Scheduling, Resource Management & Autoscaling

6.1 Requests vs Limits Senior favorite

Requests	Limits
Guaranteed minimum resources; used by the scheduler to decide which node has room	Hard ceiling; container is throttled (CPU) or killed (memory — OOMKilled) if exceeded
Under-setting causes poor scheduling decisions/overcommit	Over-setting wastes cluster capacity; under-setting causes OOMKills under load

QoS Classes derive automatically from requests/limits: **Guaranteed** (requests == limits for all containers, evicted last under node pressure), **Burstable** (requests set but less than limits, or only some containers set both), **BestEffort** (no requests/limits set at all, evicted first under node pressure).

6.2 Scheduling Constraints

- **nodeSelector** — simplest, exact label match.
- **Node affinity/anti-affinity** — richer expressions (`requiredDuringSchedulingIgnoredDuringExecution` = hard rule, `preferredDuringSchedulingIgnoredDuringExecution` = soft preference).
- **Pod affinity/anti-affinity** — schedule Pods relative to other Pods (e.g., spread replicas across nodes/zones for HA, or co-locate a cache with its consumer).
- **Taints and Toleration** — a taint on a node repels Pods unless they have a matching toleration; used to dedicate nodes (e.g., GPU nodes) to specific workloads.
- **Topology Spread Constraints** — explicitly control how Pods are distributed across zones/nodes for resilience.

6.3 Horizontal Pod Autoscaler (HPA)

Scales the *number of Pod replicas* based on observed metrics (CPU/memory via metrics-server by default, or custom/external metrics via Prometheus Adapter, e.g., queue length or request rate).

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { kind: Deployment, name: app }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }
```

6.4 Vertical Pod Autoscaler (VPA)

Adjusts a Pod's *resource requests/limits* based on historical usage, rather than replica count. Trap: HPA and VPA can conflict if both act on CPU for the same workload simultaneously (VPA changing requests can interact badly with HPA's utilization-percentage calculations) — typically use VPA on CPU/memory only in recommendation mode alongside HPA, or split responsibilities (VPA for memory, HPA for CPU/custom metrics).

6.5 Cluster Autoscaler / Karpenter

Scales the *number of nodes* in the cluster — adding nodes when Pods are unschedulable due to insufficient capacity, and removing underutilized nodes. Karpenter (newer, cloud-native) provisions right-sized nodes just-in-time per pending Pod requirements rather than scaling pre-defined node groups, often faster and more cost-efficient than the traditional Cluster Autoscaler.

6.6 Pod Disruption Budgets (PDB)

Defines the minimum number/percentage of Pods that must remain available during *voluntary* disruptions (node drains for upgrades, cluster-autoscaler scale-down) — does not protect against involuntary disruptions (node crash, OOM). Essential for HA workloads to prevent a node drain from taking down all replicas simultaneously.

7. Security & RBAC

7.1 RBAC Fundamentals

Role/ClusterRole define a set of permitted verbs (get/list/watch/create/update/delete) on resources; **RoleBinding/ClusterRoleBinding** grant that Role to a subject (User, Group, or ServiceAccount). Role+RoleBinding are namespace-scoped; ClusterRole+ClusterRoleBinding are cluster-wide (a ClusterRole can also be bound via a namespaced RoleBinding to grant cluster-defined permissions within just one namespace — a subtlety worth knowing).

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { namespace: team-a, name: pod-reader }
rules:
  - apiGroups: ["" ]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
  ---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { namespace: team-a, name: read-pods }
subjects:
  - kind: ServiceAccount
    name: ci-deployer
    namespace: team-a
roleRef: { kind: Role, name: pod-reader, apiGroup: rbac.authorization.k8s.io }
```

7.2 ServiceAccounts

Every Pod runs as a ServiceAccount (default: the namespace's `default` SA, which typically has minimal/no permissions). Best practice: create a dedicated, narrowly-scoped ServiceAccount per workload rather than relying on the default, and disable auto-mounting the SA token for Pods that don't need to talk to the API server at all (`automountServiceAccountToken: false`).

7.3 Pod Security Standards

Replaced the deprecated PodSecurityPolicy; three levels enforced via namespace labels: **Privileged** (unrestricted), **Baseline** (blocks known privilege escalations), **Restricted** (heavily hardened — non-root, no privilege escalation, dropped capabilities, seccomp required). Enforced by the built-in Pod Security Admission controller, or more flexibly by policy engines like OPA Gatekeeper/Kyverno.

7.4 Admission Controllers

Webhooks that intercept API requests before persistence: **ValidatingAdmissionWebhook** (accept/reject) and **MutatingAdmissionWebhook** (modify the request, e.g., injecting a sidecar). Used for policy enforcement (OPA Gatekeeper, Kyverno — e.g., "reject any Deployment without resource limits" or "only allow signed images").

7.5 Common Hardening Practices Senior favorite

- Run containers as non-root (`runAsNonRoot: true`), drop all Linux capabilities and add back only what's needed (`capabilities: { drop: [ALL] }`).
- Set `readOnlyRootFilesystem: true` where possible.
- Enable audit logging on the API server for compliance/forensics.
- Restrict `kubectl exec /port-forward` via RBAC in production namespaces.
- Use NetworkPolicies (default-deny) rather than relying on the flat default network.
- Scan images (Trivy/Grype) and enforce signature verification (Cosign + Kyverno/OPA) at admission time.
- Rotate and scope cloud IAM roles tied to node/service accounts (IRSA on EKS, Workload Identity on GKE) instead of static cloud credentials baked into Pods.

7.6 Secrets Encryption at Rest

Configure an `EncryptionConfiguration` on the API server with a KMS provider (cloud KMS or a Vault-based provider) so Secret data in etcd is genuinely encrypted, not just base64 — a frequent gap in real clusters that interviewers probe directly.

8. Helm & Kustomize

8.1 Helm

A package manager for Kubernetes: a **chart** bundles templated manifests plus a `values.yaml` providing per-environment parameterization. Helm tracks **releases** (an instance of a chart installed with specific values) with revision history in-cluster, enabling one-command rollback.

```
helm install myapp ./chart -f values-prod.yaml
helm upgrade --install myapp ./chart -f values-prod.yaml --set image.tag=1.4.2
helm rollback myapp 3
helm history myapp
helm template ./chart -f values-prod.yaml # render locally without installing, for review/CI diffing
```

- **Subcharts/dependencies** — a chart can depend on other charts (e.g., an app chart depending on a shared "common" library chart).
- **Hooks** — run Jobs at specific lifecycle points (`pre-install` , `post-upgrade`) for tasks like DB migrations.
- **Helm vs raw manifests** — Helm's templating (conditionals, loops, shared values) reduces duplication across environments, at the cost of added complexity/learning curve versus plain YAML.

8.2 Kustomize

A template-free alternative built into `kubect1` : define a **base** set of manifests, then **overlays** per environment that patch specific fields (image tag, replica count, resource limits) without templating syntax — pure YAML patches (strategic merge or JSON patch).

```
# base/kustomization.yaml
resources: [deployment.yaml, service.yaml]

# overlays/prod/kustomization.yaml
resources: [.././base]
patches:
- target: { kind: Deployment, name: app }
  patch: |-
    - op: replace
      path: /spec/replicas
      value: 5
```

8.3 Helm vs Kustomize Senior favorite

Helm	Kustomize
Templating language (Go templates) — powerful but can become hard to read/debug ("YAML generating YAML")	Pure declarative patches on plain YAML — simpler mental model, no templating logic to debug
Strong packaging/versioning/release-history model; large public chart ecosystem	No packaging/release concept built in; typically paired with a GitOps tool for release tracking
Better for distributing reusable, configurable third-party software (e.g., installing Prometheus)	Better for managing your own app's environment-specific overlays with minimal abstraction

Many teams use both: Helm to install third-party/vendor charts, Kustomize (or Helm itself) to manage overlays of their own application manifests. ArgoCD natively supports both as templating backends.

9. GitOps Integration (ArgoCD/Flux) & CI/CD

9.1 Why GitOps for Kubernetes Senior favorite

Instead of a CI pipeline running `kubectl apply / helm upgrade` directly against the cluster (push-based, requiring the CI system to hold cluster credentials), a GitOps controller runs *inside* the cluster and continuously pulls/reconciles state from a Git repo (pull-based). Benefits: cluster credentials never leave the cluster, all changes are auditable via Git history, and drift (manual `kubectl edit`) is automatically detected and can be self-healed back to the Git-defined state.

9.2 ArgoCD Core Concepts

- **Application CRD** — maps a Git repo path (plus revision) to a destination cluster/namespace.
- **Sync status:** Synced / OutOfSync — whether live state matches Git.
- **Health status:** Healthy / Progressing / Degraded / Suspended — derived from resource-specific health checks (e.g., a Deployment is Healthy once all replicas are ready).
- **Sync policy:** manual (click to sync) vs automated; `selfHeal: true` reverts manual drift automatically; `prune: true` deletes resources removed from Git.
- **App-of-Apps pattern** — a parent Application manages many child Applications, used to manage dozens/hundreds of microservices from one root.
- **Sync waves/hooks** — control ordering (e.g., apply a CRD before the resources using it) and run Jobs at specific sync phases (PreSync/PostSync).

9.3 Flux

Similar GitOps model via separate controllers: source-controller (watches Git/Helm repos), kustomize-controller (applies Kustomize overlays), helm-controller (manages Helm releases) — a more modular, Unix-philosophy design versus ArgoCD's more monolithic (but UI-rich) architecture.

9.4 Progressive Delivery on Top of GitOps

Argo Rollouts and **Flagger** replace the plain Deployment resource with a Rollout/Canary CRD that shifts traffic gradually and queries a metrics provider (Prometheus, Datadog) at each step, automatically pausing or rolling back if error rate/latency regresses — giving you canary/blue-green deployment with automated safety, layered on top of the GitOps reconciliation model.

9.5 Where CI Fits

```
Developer pushes app code
→ CI (GitHub Actions/GitLab CI/Jenkins) builds, tests, scans, pushes image:sha
→ CI updates image tag in a separate GitOps/config repo (or Helm values)
→ ArgoCD/Flux detects the Git change and reconciles the cluster
→ Argo Rollouts/Flagger progressively shifts traffic, monitoring live metrics
→ Auto-promote to 100% or auto-rollback based on metric thresholds
```

Key interview point: the app CI pipeline never directly touches the cluster in this model — it only ever writes to Git.

10. Observability & Health Checks

10.1 The Three Probes Senior favorite

Probe	Purpose	On failure
Liveness	Is the container alive/functioning?	kubelet kills and restarts the container
Readiness	Is the container ready to serve traffic right now?	Pod is removed from Service endpoints (no restart) until it passes again
Startup	Has the (possibly slow-starting) app finished initializing?	Liveness/readiness probes are held off until startup succeeds, preventing premature kills of slow-starting apps

Classic trap: setting liveness probe timing too aggressively causes restart loops on a healthy-but-slow app; conflating liveness with readiness (using the same endpoint/threshold for both) can cause a temporarily-overloaded-but-alive Pod to be killed and restarted instead of just being drained from traffic.

10.2 Metrics

metrics-server provides basic CPU/memory metrics for `kubectl top` and HPA; for real observability, **Prometheus** scrapes richer application and cluster metrics (via exporters/instrumentation), visualized in **Grafana**, with **Alertmanager** handling alert routing. **kube-state-metrics** exposes the state of Kubernetes objects themselves (Deployment replica counts, Pod phase, etc.) as Prometheus metrics, distinct from cAdvisor/node-level resource metrics.

10.3 Logging

Kubernetes itself doesn't provide long-term log storage — container stdout/stderr are captured by the container runtime and typically shipped by a node-level DaemonSet agent (Fluentd/Fluent Bit/Vector) to a centralized backend (ELK/EFK, Loki, cloud logging service).

10.4 Tracing

Distributed tracing (OpenTelemetry, Jaeger) becomes essential once a request spans multiple services/Pods — correlating a single user request across the mesh to find where latency/errors originate.

10.5 Events & Debugging Signals

`kubectl get events --sort-by=.lastTimestamp` and `kubectl describe pod` surface scheduler decisions, image pull failures, probe failures, and OOM kills — usually the fastest first step in any troubleshooting flow, before diving into logs.

11. Common Errors & Troubleshooting Playbook

Error: `CrashLoopBackOff`

Fix: The container starts and then exits repeatedly. Check `kubectl logs <pod> --previous` for the actual crash reason (application error, missing config/env var, failing DB connection). Common causes: bad entrypoint/command, missing ConfigMap/Secret key, or the app crashing because a dependency (DB, downstream service) isn't reachable yet.

Error: `ImagePullBackOff` / `ErrImagePull`

Fix: Wrong image name/tag, private registry without an `imagePullSecrets` configured on the Pod/ServiceAccount, or registry rate-limiting/network policy blocking the pull. Verify with `kubectl describe pod` for the exact pull error message.

Error: `OOMKilled`

Fix: Container exceeded its memory `limit`. Either the limit is set too low for real usage (check actual usage via `kubectl top pod` / Prometheus and raise the limit), or there's a genuine memory leak in the application — distinguish by watching memory usage trend over time, not just at the moment of the kill.

Error: Pod stuck in `Pending`

Fix: `kubectl describe pod` shows the scheduler's reason — usually insufficient CPU/memory on any node, no node matching a nodeSelector/affinity rule, an unbound PVC (no matching PV/StorageClass), or a taint with no matching toleration. If it's a capacity issue, check whether Cluster Autoscaler/Karpenter should have added a node and why it didn't (e.g., a node group max size reached).

Error: Service has no endpoints / `curl` to the service times out from inside the cluster.

Fix: `kubectl get endpoints <svc>` — if empty, the Service's label selector doesn't match any Pod's labels (typo, or Pods aren't Ready so they're excluded from endpoints), or the target port doesn't match the container's actual listening port.

Error: Rolling update hangs, never completes ("1 of 3 updated replicas available").

Fix: New Pods are failing readiness probes (check logs/describe on the new ReplicaSet's Pods specifically, not the old ones), `maxUnavailable: 0` combined with insufficient cluster capacity to schedule the surge Pods, or a PodDisruptionBudget blocking eviction of old Pods during the rollout.

Error: `kubectl apply` succeeds but nothing changes in the running Pods.

Fix: Only the Pod template hash changed if the Deployment spec's Pod template actually changed — editing a referenced ConfigMap/Secret in place does NOT trigger a rollout by itself (Deployments only roll on changes to their own spec). Fix: use a checksum annotation on the Pod template (common Helm pattern: `checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`) so config changes are reflected in the Pod template hash and trigger a real rollout.

Error: Node shows `NotReady`.

Fix: kubelet on that node stopped reporting (crashed, network partition, resource exhaustion on the node itself, or clock skew affecting cert validity). Check node conditions (`kubectl describe node`), kubelet logs on the node, and container runtime health.

Error: ArgoCD app shows `OutOfSync` repeatedly right after every sync ("flapping").

Fix: Usually a mutating admission webhook or a controller (e.g., HPA adjusting replica count) is changing a field that Git doesn't match, causing ArgoCD to see permanent drift. Fix by adding an `ignoreDifferences` rule in the Application spec for fields legitimately managed outside Git (like `spec.replicas` when HPA controls it).

Error: `etcd` or API server latency spikes, cluster feels sluggish under load.

Fix: Common root causes: too many objects/watch events (excessive ConfigMap/Secret churn, very large numbers of Pods/CRDs), insufficient etcd disk IOPS, or a controller in a hot reconcile loop. Check API server request latency metrics and etcd's own health/latency metrics first.

11.1 General Debugging Method

1. `kubectl get events --sort-by=.lastTimestamp` and `kubectl describe` the resource first — often tells you the exact reason before

you touch logs.

2. `kubectl logs` (and `--previous` for a crashed container) for application-level detail.
3. Check "what changed" — recent deploy, ConfigMap edit, node pool change, cluster upgrade.
4. Check resource pressure (node and Pod level) via `kubectl top` / Prometheus.
5. Isolate scope: one Pod, one node, one namespace, or cluster-wide?
6. Fix root cause, then add a regression guard (probe tuning, PDB, resource limit, alert) so it can't silently recur.

12. Tricky Questions & Traps

Q: Are Kubernetes Secrets encrypted?

No — by default — they're only base64-encoded, which is trivially reversible. Real encryption at rest requires explicitly configuring an EncryptionConfiguration with a KMS provider on the API server. Many candidates incorrectly assume "Secret" means "encrypted".

Q: If you edit a ConfigMap that's mounted as an environment variable, does the running Pod pick up the change?

No — env vars are only read once at container start; only file-mounted ConfigMaps update in place (with a propagation delay), and even then the application itself must watch/reload the file — env-var-based config always needs a Pod restart to take effect.

Q: Does a Pod Disruption Budget protect against a node crashing unexpectedly?

No — PDBs only govern voluntary disruptions (drains, scale-down, cordoning for maintenance); an involuntary disruption like a node hardware failure or OOM ignores the PDB entirely. This distinction is a frequent trap.

Q: Does ReadWriteOnce mean only one Pod can mount the volume?

No — RWO means the volume can be mounted read-write by a single *node*, not a single Pod; multiple Pods co-located on that same node can still mount it simultaneously. If you truly need single-Pod exclusivity, use ReadWriteOncePod.

Q: Is a namespace a security boundary?

Partially, and easy to overstate — namespaces scope RBAC and resource quotas, but by default there's no network isolation between namespaces (flat network) unless NetworkPolicies enforce it, and several core resources (Nodes, PersistentVolumes, ClusterRoles) are cluster-scoped and not namespace-isolated at all.

Q: If your Deployment's rollout strategy shows all replicas "Available", is the rollout guaranteed safe?

"Available" only reflects the readiness probe passing for the configured minimum time — it says nothing about actual business-logic correctness, error rates under real production load, or downstream integration issues; this is exactly why canary analysis with real metrics (Argo Rollouts/Flagger) exists on top of basic rollout status.

Q: Does installing a service mesh automatically make inter-service traffic encrypted and secure?

Not automatically — mTLS must be explicitly enabled/enforced in the mesh's policy (e.g., Istio's PeerAuthentication set to STRICT); many meshes default to permissive mode where plaintext is still accepted, silently leaving traffic unencrypted despite the mesh being "installed".

Q: Can you scale a StatefulSet exactly like a Deployment, in any order, safely?

No — StatefulSets scale up/down and roll updates in strict ordinal order by default (app-0 before app-1, etc.) precisely because of the stable-identity guarantee; treating it like an interchangeable Deployment can break ordering assumptions many stateful apps (e.g., cluster-forming databases) rely on.

Q: Is "kubectl apply succeeded" proof that the desired state is actually running?

No — `apply` only confirms the API server accepted and stored the new desired state; it says nothing about whether the scheduler could place the Pods, whether images pulled successfully, or whether probes are passing. Always verify actual rollout status (`kubectl rollout status`) separately.

Q: Does the Horizontal Pod Autoscaler prevent an outage from a traffic spike?

Not by itself — HPA reacts to metrics with a lag (metrics collection interval + scale-up time + new Pod startup time), and it can't create capacity faster than the Cluster Autoscaler/Karpenter can add nodes if existing nodes are already full; a sudden enough spike can still cause a brief outage before scaling catches up, which is why pre-provisioned headroom or predictive scaling matters for spiky workloads.

13. Expected Interview Q&A (Basic → Advanced)

13.1 Basic Level

Q: What is a Pod?

The smallest deployable unit in Kubernetes — one or more containers sharing network namespace and optionally storage, scheduled together onto the same node.

Q: What is the difference between a Deployment and a ReplicaSet?

A ReplicaSet just maintains a desired number of Pod replicas; a Deployment manages ReplicaSets and adds declarative rolling update and rollback capability on top — you interact with Deployments in practice, not ReplicaSets directly.

Q: What's the difference between a ConfigMap and a Secret?

Same underlying mechanism (key-value data injectable into Pods), but Secrets are intended for sensitive data; note that Secrets are only base64-encoded by default, not encrypted, unless encryption at rest is separately configured.

Q: What is a Service and why is it needed?

Pods are ephemeral and get new IPs when recreated; a Service provides a stable virtual IP/DNS name and load-balances traffic across the current set of matching Pods via label selectors.

Q: What is the difference between liveness and readiness probes?

Liveness determines whether to restart a container (is it stuck/dead); readiness determines whether to send it traffic right now (is it ready), without restarting it.

Q: What is a namespace?

A logical partition within a cluster providing a scope for names, RBAC, and resource quotas — not a full network or security isolation boundary by default.

13.2 Intermediate Level

Q: How does Kubernetes decide which node to schedule a Pod on?

The scheduler filters nodes that satisfy hard constraints (resource requests, node selectors, taints/tolerations, required affinity rules) then scores/ranks the remaining feasible nodes (based on preferred affinity, resource balance, etc.) and picks the best-scoring one.

Q: How would you safely roll out a change to a database schema alongside an application deployment?

Use the expand/contract migration pattern: deploy a backward-compatible schema change first (additive only), then roll out application code that uses it, and only in a later separate step remove any now-unused old columns/tables — ensuring the rollout (and any rollback) never leaves incompatible code and schema running together.

Q: How do you debug a Pod that's stuck in Pending?

`kubectl describe pod` to read the scheduler's FailedScheduling event reason — typically insufficient node resources, an unsatisfied affinity/taint rule, or an unbound PVC; then check cluster autoscaler logs if capacity should have been added automatically.

Q: What's the difference between Helm and Kustomize, and when would you choose one over the other?

Helm templates YAML with a full templating language and packages/versions releases; Kustomize patches plain YAML overlays without templating syntax. Helm suits installing third-party/vendor software and complex parameterized charts; Kustomize suits managing your own app's environment-specific overlays with a simpler mental model.

Q: How do you prevent a bad deployment from taking down an entire service?

Rolling update with `maxUnavailable` tuned conservatively, readiness probes gating traffic to new Pods, a PodDisruptionBudget protecting minimum availability, and ideally canary analysis (Argo Rollouts/Flagger) automatically halting/rolling back on metric regression before 100% of traffic shifts.

13.3 Advanced / Senior Level

Q: Design the deployment architecture for a multi-tenant SaaS platform running on Kubernetes, where tenants must be strongly isolated.

Talk through: namespace-per-tenant as a baseline with strict RBAC and default-deny NetworkPolicies between tenant namespaces; ResourceQuotas/LimitRanges per namespace to prevent noisy-neighbor resource exhaustion; for stronger isolation than namespaces

provide, consider dedicated node pools per tier (or per large tenant) with taints/tolerations, or separate clusters entirely for the highest-sensitivity tenants; centralized but tenant-scoped observability (metrics/logs tagged by tenant, with RBAC-restricted dashboards); and admission policies (OPA/Kyverno) enforcing tenant-specific guardrails automatically rather than relying on manual review.

Q: How would you design zero-downtime cluster upgrades for a production Kubernetes cluster?

Talk through: upgrade control plane first (usually managed by the cloud provider with minimal disruption for managed K8s), then upgrade node pools by creating a new node pool on the target version and cordoning/draining old nodes gradually (respecting PodDisruptionBudgets), verify workload health at each batch, and keep the ability to roll back the node pool if workloads show regressions; also address any deprecated API versions in manifests before the upgrade using tools like `kubectl deprecations` or `pluto`.

Q: A production incident: pods are being OOMKilled intermittently under normal-looking load. Walk through your investigation.

Talk through: confirm via `kubectl describe pod` /events that it's genuinely OOMKilled (reason + exit code 137), check actual memory usage trend over time via Prometheus rather than a single snapshot to distinguish a leak from a legitimate spike, correlate with recent deploys/config changes, check whether the memory limit was set based on stale/underestimated usage data, and check for GC behavior/language-runtime-specific memory patterns (e.g., JVM heap vs container memory limit mismatches) that are a very common root cause of "OOMKilled despite looking fine".

Q: How do you enforce that all workloads in a cluster meet security/compliance baselines without relying on manual review?

Enforce via policy-as-code at admission time: OPA Gatekeeper or Kyverno policies requiring resource limits, non-root users, no privileged containers, mandatory labels, and signed images only; combine with Pod Security Admission at the Restricted level for namespace-wide baseline enforcement, and CI-side scanning (Checkov/tfsec/Trivy) as a pre-merge gate so violations are caught before they even reach the cluster, not just blocked at admission.

Q: How would you reduce the blast radius of a compromised Pod in your cluster?

Least-privilege RBAC/ServiceAccount per workload with token auto-mount disabled where unnecessary, default-deny NetworkPolicies limiting lateral movement, non-root/read-only-root-filesystem/dropped-capabilities container security context, node-level isolation (dedicated node pools) for sensitive workloads, runtime security monitoring (Falco) to detect anomalous syscalls, and image signing/admission enforcement to prevent unauthorized images from running in the first place.

14. System Design Style Questions

Structure: clarify requirements → propose cluster/workload architecture → call out trade-offs → mention security/observability/rollback.

Q: Design a highly available Kubernetes deployment for a stateless API expecting unpredictable traffic spikes.

Talk through: Deployment with replicas spread across zones (topology spread constraints/anti-affinity), HPA on request-rate or CPU with headroom, Cluster Autoscaler/Karpenter for node-level scaling, a PodDisruptionBudget to survive voluntary node maintenance, readiness probes gating traffic during scale-up, and a service mesh or Ingress with circuit-breaking to protect downstream dependencies during a spike.

Q: Design a Kubernetes-based CI/CD platform for 200 microservices with independent teams and release cadences.

Talk through: GitOps (ArgoCD App-of-Apps) as the deployment backbone so every team's pipeline only needs to update Git, not touch the cluster directly; a platform team owning shared Helm charts/Kustomize bases and admission policies so teams get guardrails "for free"; namespace-per-service (or per-team) with RBAC boundaries; progressive delivery (Argo Rollouts) as a shared capability rather than each team reinventing canary logic; and centralized observability with per-team dashboards.

Q: Design a disaster recovery strategy for a stateful workload (e.g., PostgreSQL) running on Kubernetes.

Talk through: StatefulSet with per-replica PVCs on a CSI driver supporting snapshots, automated periodic volume snapshots plus off-cluster backup (e.g., to object storage) for cross-region recovery, a documented/tested restore runbook (not just backups — actually rehearsed restores), replication to a standby (often via an operator like Patroni/CloudNativePG rather than hand-rolled StatefulSet logic for databases), and clear RTO/RPO targets driving the backup frequency and replication topology chosen.

15. Behavioral / Experience-Based Questions

Prepare STAR-format stories from your own project history for each:

- "Describe a production Kubernetes incident you diagnosed and resolved. What was the root cause, and what guardrail did you add afterward?"
- "Tell me about a time you improved resource utilization or cost on a Kubernetes cluster."
- "Describe migrating a workload from a traditional VM-based deployment to Kubernetes — what was hardest?"
- "Tell me about a security gap you found or closed in a Kubernetes environment (RBAC, network policy, secrets handling, etc.)."
- "How have you mentored a team on adopting GitOps or Kubernetes best practices?"
- "Describe the most complex Kubernetes upgrade or migration you've led."

16. Quick Reference Cheat Sheet (kubectl)

16.1 Common Commands

```
# Inspect
kubectl get pods -o wide
kubectl describe pod <pod>
kubectl logs <pod> -c <container> --previous
kubectl get events --sort-by=.lastTimestamp
kubectl top pod / kubectl top node

# Rollouts
kubectl rollout status deployment/app
kubectl rollout undo deployment/app
kubectl rollout history deployment/app
kubectl rollout restart deployment/app

# Debugging
kubectl exec -it <pod> -- /bin/sh
kubectl port-forward svc/app 8080:80
kubectl debug <pod> -it --image=busybox

# Scaling
kubectl scale deployment/app --replicas=5
kubectl autoscale deployment/app --min=3 --max=10 --cpu-percent=70

# RBAC check
kubectl auth can-i create pods --namespace team-a --as=system:serviceaccount:team-a:ci-deployer

# Context/namespace
kubectl config use-context <context>
kubectl config set-context --current --namespace=<ns>

# Helm
helm upgrade --install app ./chart -f values-prod.yaml
helm rollback app 2

# ArgoCD
argocd app sync my-app
argocd app get my-app
argocd app rollback my-app
```

16.2 One-Line Definitions to Have Ready

Reconciliation loop	Controller pattern: watch desired state, compare to actual, act to close the gap
QoS class	Guaranteed/Burstable/BestEffort, derived from resource requests/limits
PDB	Minimum availability guarantee during voluntary disruptions only
CSI	Standard interface letting any storage vendor plug into Kubernetes
RWO vs RWOP	Per-node exclusive vs true per-Pod exclusive volume access
GitOps	Cluster-side controller pulls/reconciles from Git; credentials never leave the cluster
Admission controller	Webhook that validates/mutates API requests before they're persisted
HPA vs VPA vs Cluster Autoscaler	Replica count vs per-Pod resource sizing vs node count

16.3 Day-Before-Interview Checklist

- Re-read section 7 (Security/RBAC) and section 9 (GitOps) — these differentiate senior candidates.
- Be ready to explain the difference between liveness/readiness/startup probes from memory.
- Rehearse 2-3 STAR stories (section 15).
- Review the tricky Q&A (section 12) — several are common "gotcha" traps.
- Know exactly how your own recent projects handle storage, secrets, and rollout/rollback.